

Autonomous Lottery Ticket Discovery

面向 Compressibility Frontier 的长期自主模型压缩

核心问题：How small can a model make itself?

不预先指定目标稀疏率或模型大小，而是让模型通过长期的“压缩—诊断—恢复—继承”闭环，不断寻找更小、但仍能继承原模型能力的后代模型，最终自主发现其 **Compressibility Frontier**。

1. Research Motivation

传统模型压缩通常预先给定压缩预算，例如：

- 50% sparsity；
- 4-bit / 3-bit quantization；
- 固定参数量；
- 固定 layer-wise pruning ratio。

然后求解：

$$\max_{M'} \text{Perf}(M')$$

subject to

$$C(M') \leq B,$$

其中 B 是人为设定的 compression budget。

本项目希望改变这一问题：

不提前告诉模型需要压缩到多小，而是让模型不断尝试：我还能不能再小一点？

模型形成一个长期演化过程：

$$M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow \cdots \rightarrow M_T.$$

每个后代模型 M_{t+1} 只有满足能力继承约束后才能被接受：

$$\text{Perf}(M_{t+1}) \geq \text{Perf}(M_0) - \delta,$$

或者使用更严格的局部约束：

$$\text{Perf}(M_{t+1}) \geq \text{Perf}(M_t) - \epsilon.$$

当系统已经无法在有限 search/recovery budget 内产生一个更小且满足能力约束的后代时：

$$M_T \rightarrow M_{T+1} \text{ fails,}$$

我们认为 M_T 到达当前条件下的经验：

Compressibility Frontier

2. 从 Lottery Ticket 到 Autonomous Ticket

经典 Lottery Ticket Hypothesis 指出，dense network 中可以存在能够独立训练并达到接近原网络性能的稀疏子网络；经典 IMP 通过反复 train-prune-rewind-retrain 搜索这样的子网络。

本项目不直接声称寻找经典意义上的 Lottery Ticket，而是借鉴其核心思想：

不要一次决定最终压缩结构，而是通过连续搜索逐渐发现 viable subnetwork。

因此定义：

Autonomous Ticket

从当前 parent model M_t 中产生的一个更小的 candidate descendant：

$$T_t = (M'_t, c_t),$$

其中 c_t 表示：

- pruning region；
- pruning ratio；
- quantization precision；
- regrowth operation；
- recovery budget。

如果经过有限恢复以后：

$$M'_t \xrightarrow{\text{Recovery}} M_{t+1}$$

能够满足：

$$M_{t+1} \in \mathcal{A},$$

其中 $\mathcal{A}$ 为 acceptable capability region，则认为：

$$T_t$$

是一张 **winning autonomous ticket**。

最终问题变成：

Autonomously discover a sequence of viable compressed descendants.

3. 与 Compression Frontier / BigBang 方案的统一

前面的 Compression Frontier 方案解决的是：

模型被压缩以后，到底损坏了什么能力，以及什么数据最适合恢复这些能力？

对于 parent M_t 和 compressed child M'_{t+1} ，定义：

$$G_t(x) = L_{M'_{t+1}}(y_t|x) - L_{M_t}(y_t|x),$$

其中：

$$y_t = M_t(x).$$

如果：

$$G_t(x) \gg 0,$$

表示 parent 对该任务仍有较强建模能力，但 child 因压缩明显退化。

因此：

$$D_t^{gap} = \text{TopK}_x G_t(x)$$

描述本轮 compression-induced capability frontier。

BigBang 提供的是另一层长期闭环思想：

$$CapabilityGap \rightarrow Generator \rightarrow Critic \rightarrow Training \rightarrow RealEvaluation \rightarrow NextRound.$$

其生成策略会随着真实 capability gap 和训练效果不断改变。

因此，本项目将三者统一：

$$Diagnose \rightarrow SearchTicket \rightarrow Compress \rightarrow ProfileGap \rightarrow Recover \rightarrow Evaluate \rightarrow Inherit$$

其中：

- **Lottery Ticket**：提供 iterative subnetwork search；

- **Compression Frontier**：提供压缩损伤诊断；
- **BigBang-style evolution**：根据上一轮反馈改变下一轮搜索与数据生成；
- **Self-Distillation**：parent 自动帮助 compressed child 恢复能力。

4. 核心 Research Questions

RQ1：模型的 Compressibility Frontier 是否可以被自主发现？

比较：

$$M_0 \rightarrow M_{50\%}$$

与：

$$M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow \dots$$

Hypothesis H1

在相同 performance constraint 下：

$$C(M_T^{adaptive}) < C(M^{fixed}).$$

即 adaptive iterative search 可以找到比人为指定压缩策略更靠前的 Pareto frontier。

RQ2：Compression-induced capability loss 是否具有结构？

研究：

$$P(G_t | task, layer, length, compression policy).$$

Hypothesis H2

压缩损伤并不是均匀的，而会集中在特定：

- capability；
- reasoning pattern；
- context length；
- network region；
- compression strategy。

因此不同压缩状态具有不同的：

Compression Frontier

RQ3：Parent 能否根据 Child 的 failure 自动产生 recovery data？

构造：

$$D_t^{gap} = \{x : M_t \text{ strong}, M'_{t+1} \text{ weak}\}.$$

然后：

$$D_t^{syn} = \text{Generator}(M_t, D_t^{gap}).$$

Hypothesis H3

$$\text{Recovery}(M'_{t+1}, D_t^{syn}) > \text{Recovery}(M'_{t+1}, D_{random}).$$

即恢复数据应该是：

$$D^* = f(M_t, M_{t+1}, c_t),$$

而不是固定通用数据。

RQ4：Child 能否接替 Parent 成为下一代 Controller？

已有 agent-guided pruning 更接近：

$$A_{large} \rightarrow M_0 \rightarrow M_1 \rightarrow M_2,$$

即外部 controller 保持不变。近期已有工作已经利用 foundation model、layer sensitivity、历史 pruning outcome 和 rollback 来控制 iterative pruning，因此单纯的“LLM agent 自动选 layer”本身不足以构成本工作的核心创新。

我们研究：

$$M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_3$$

其中被接受的 M_t 自己承担：

1. task model；
2. compression diagnostician；
3. data generator；
4. compression controller。

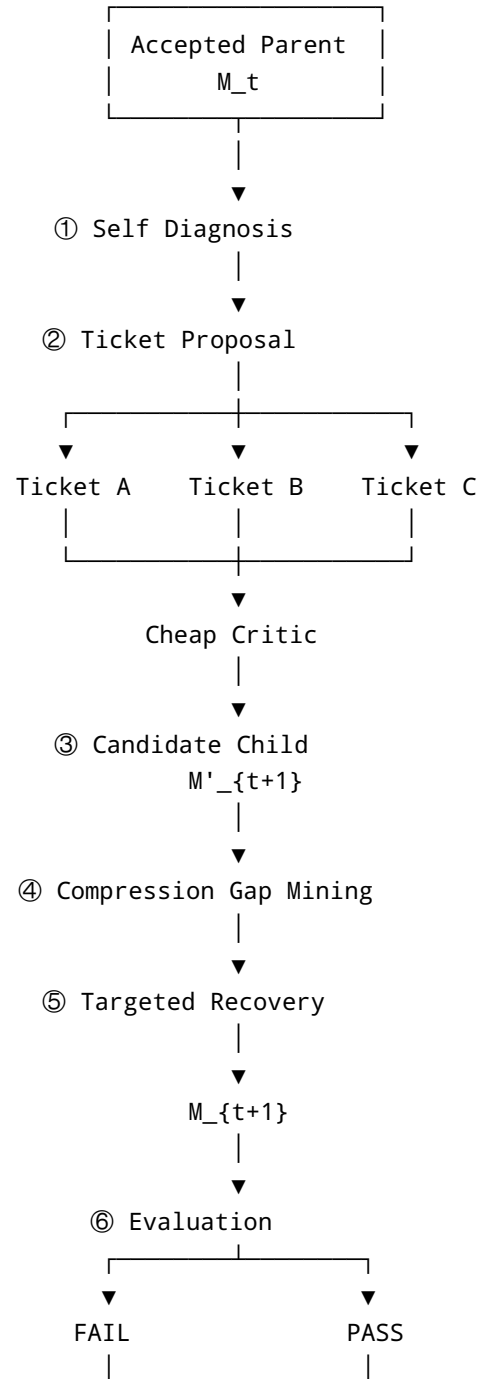
由此产生一个新的问题：

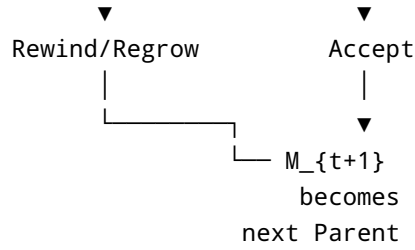
$$\text{Self-Governance Frontier}$$

即：

模型在失去任务能力之前，会不会先失去可靠压缩自己的能力？

5. Overall Framework





6. Module 1 : Self Diagnosis

每轮构建三个 profile。

6.1 Structural Sensitivity

对于 layer/module l :

$$S_l = f(|W_l|, Activation_l, Gradient_l, ReconstructionError_l).$$

第一版直接使用：

- magnitude ;
- Wanda ;
- SparseGPT ;
- gradient sensitivity。

这里不优先设计新的 **pruning importance score**。

重点是：

如何利用长期反馈决定下一轮压缩策略。

6.2 Capability Sensitivity

使用 candidate pool :

$$X = \{x_1, \dots, x_N\}.$$

比较 parent 与 candidate child :

$$G_t(x) = L_{child}(y_t|x) - L_{parent}(y_t|x).$$

形成：

```
math          gap = ...
knowledge     gap = ...
code          gap = ...
reasoning     gap = ...
long context  gap = ...
instruction    gap = ...
```

从而知道：

哪些能力最接近当前模型的 compression boundary。

6.3 Historical Sensitivity

维护：

$$H_t = \{a_i, \Delta C_i, \Delta Perf_i, G_i, Recovery_i, Rollback_i\}_{i < t}.$$

Controller 不仅知道：

当前 layer 12 看起来不敏感，

还知道：

generation 4 曾经剪 layer 12 以后 math loss 很低。

因此系统会逐渐积累 compression experience。

7. Module 2 : Adaptive Ticket Proposal

单步 action 定义为：

$$a_t = (region, operator, ratio, recoveryBudget).$$

例如：

```
region: MLP layers 8-14
operator: structured_pruning
ratio: 0.05
recovery_budget: 200 LoRA steps
```

第一阶段：

$$operator \in \{Prune, Regrow, Hold\}.$$

后续扩展：

$$operator \in \{Prune, Regrow, Quantize, LowRank, Recover\}.$$

与固定 IMP 最大区别是：

$$a_t = \pi(Profile_t, H_t),$$

而不是：

$$s_{t+1} = s_t + 5\%.$$

8. Module 3 : Lottery Ticket Search

每轮不是只生成一个 candidate, 而是：

$$\mathcal{T}_t = \{T_t^1, T_t^2, \dots, T_t^K\}.$$

例如：

Ticket A
MLP 8-14 additional 5% pruning

Ticket B
Attention 3-9 additional 8% pruning

Ticket C
Regrow MLP-21
+ prune MLP-6 additional 10%

首先使用廉价 proxy：

$$Q(T) = CompressionGain - \lambda_1 PPLLoss - \lambda_2 \widehat{CapabilityLoss}.$$

只对 Top- k ticket 做昂贵的：

- full benchmark ;
- recovery ;
- downstream validation。

形成：

$$Generator \rightarrow CheapCritic \rightarrow RealUtility.$$

9. Module 4 : Compression Frontier Profiling

对于：

$$M_t \rightarrow M'_{t+1},$$

计算：

$$G_t(x) = L_{M'_{t+1}}(y_t|x) - L_{M_t}(y_t|x).$$

不仅计算：

$$E_x[G_t(x)],$$

还计算：

$$E[G_t|task],$$

$$E[G_t|length],$$

$$E[G_t|difficulty].$$

最终得到：

这一轮压缩具体损害了什么，而不仅仅是损失了多少平均 accuracy。

10. Module 5 : Parent-to-Child Self Recovery

Step 1 : Frontier Mining

选择：

$$D_t^{gap} = TopK(G_t(x)).$$

控制：

- sequence length ;
 - domain ratio ;
 - difficulty ;
 - diversity。
-

Step 2 : Failure Abstraction

Parent 不直接改写失败样本，而先总结：

Compression-induced failures:

1. multi-step numerical state tracking
2. long dependency reasoning
3. nested logical constraints
4. rare factual recall

得到：

$$Z_t = \text{Abstract}(D_t^{\text{gap}}).$$

Step 3 : Synthetic Frontier Data

生成：

$$D_t^{\text{syn}} = \text{Generator}(M_t, Z_t).$$

候选数据至少满足：

Parent competence

$$\text{Conf}_{M_t}(x) > \tau.$$

Compression gap

$$G_t(x) > \gamma.$$

Verifiability

答案可自动验证。

Diversity

不能只是原问题的 paraphrase。

11. Module 6 : Recovery Operators

Recovery 按计算成本分层。

Level 0

No recovery。

用于确定原始 compression damage。

Level 1

Calibration / reconstruction。

Level 2

Frontier-data LoRA。

Level 3

Parent-to-child self-distillation :

$$M_t \rightarrow M_{t+1}.$$

例如：

$$\mathcal{L} = \mathcal{L}_{task} + \alpha KL(P_{M_t} || P_{M_{t+1}}).$$

必须显式记录：

$$Cost_{recovery}.$$

否则使用大量训练得到较小模型，并不能证明 autonomous compression 是高效的。

12. Module 7 : Acceptance、Rewind 与 Regrowth

不能只用平均 benchmark。

定义能力向量：

$$\mathbf{P}(M) = [PPL, Math, Knowledge, Reasoning, Code, Instruction, LongContext].$$

Acceptance：

$$Aggregate(M_{t+1}) \geq Aggregate(M_0) - \delta,$$

同时：

$$P_j(M_{t+1}) \geq P_j(M_0) - \delta_j.$$

这样可以防止：

math +2, 但 knowledge -10

被平均分掩盖。

12.1 Ancestral Rewinding

失败：

$$M'_{t+1} \notin \mathcal{A}$$

则直接：

$$M'_{t+1} \rightarrow M_t.$$

即永远从最后一个 **accepted ancestor** 继续搜索。

这对应经典 pruning 中 rewinding 的思想，但这里 rewind 的对象是上一代可靠模型，而不是必须回到初始化。经典工作表明 rewinding 到较早 checkpoint 可以显著提高 pruning 稳定性。

12.2 Regrowth

允许：

$$1 \rightarrow 0$$

同时允许：

$$0 \rightarrow 1.$$

也就是说：

$$m_{t+1} \not\subseteq m_t.$$

如果某结构被删除以后：

- capability gap 很大；
- recovery 很贵；
- 历史反复失败；

则恢复它，再从其他位置获得 compression gain。

RigL 已经证明动态改变 sparse connectivity 可以优于固定 topology；本项目借用这一思想，但 regrowth decision 由 capability loss 和长期 lineage feedback 驱动。

13. Autonomous Stopping

系统不预设：

target sparsity = 60%

定义当前 parent 周围的 action space：

$$\mathcal{A}_t = \{a_t^1, \dots, a_t^K\}.$$

如果连续 R 个 search round：

$$\forall a \in \mathcal{A}_t,$$

都有：

$$Perf(Recover(Compress(M_t, a))) < Perf_{min},$$

或者：

$$Cost_{recovery} > B_{recover},$$

则停止：

$$M_t \approx Empirical\ Compressibility\ Frontier.$$

因此：

$$Frontier = F(M_0, TaskDistribution, QualityTolerance, RecoveryBudget).$$

14. Optimization Objective

总体目标：

$$\min C(M_T)$$

subject to：

$$Perf(M_T) \geq Perf(M_0) - \delta,$$

以及：

$$\sum_t Cost_{recover}(t) \leq B.$$

单步 reward：

$$R_t = \log \frac{C(M_t)}{C(M_{t+1})} - \lambda[\Delta Perf_t - \epsilon]_+ - \mu Cost_{recover}(t) - \nu Cost_{search}(t).$$

因此本项目优化的不是：

固定 50% sparsity 下谁 accuracy 最高，

而是：

固定允许的能力损失和 search budget 下，模型最终能够自己走到多小。

15. Experimental Roadmap

整个项目分成四阶段，而不是直接实现完整 Agent。

Stage A：Does a Compressibility Frontier Exist?

Model

首先：

Qwen-2.5-3B.

Compression

先固定一种：

- Wanda-style pruning；

或者：

- structured MLP pruning。

Procedure

固定：

$$\Delta s = 5\%.$$

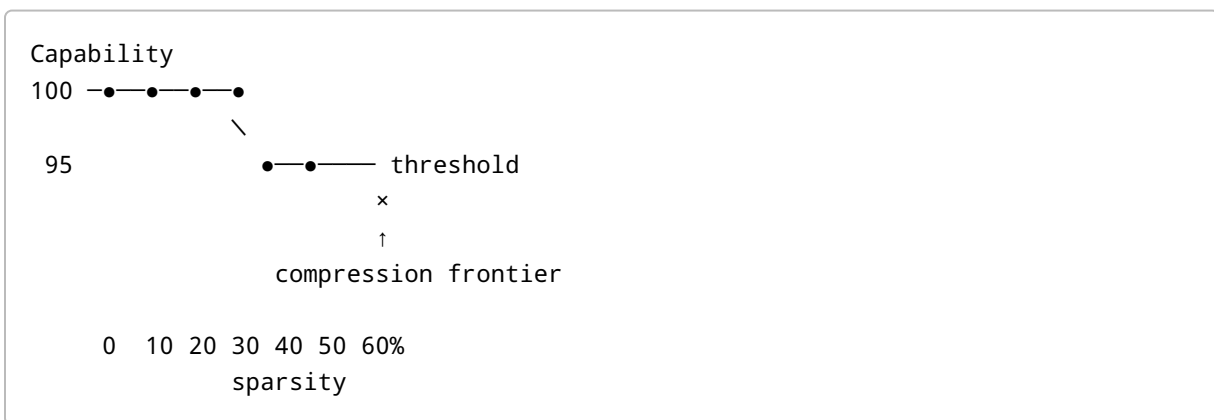
运行：

$$M_0 \rightarrow M_5 \rightarrow M_{10} \rightarrow \dots$$

观察：

1. 是否存在明显 performance cliff；
2. 哪些 capability 最先下降；
3. 不同 capability 是否具有不同 frontier。

Figure



16. Stage B : Adaptive Lottery Ticket Search

暂时不加入 synthetic data。

比较：

B0 : One-shot

直接达到某 sparsity。

B1 : Fixed Iterative

固定：

$$+5\%$$

逐步剪枝。

B2 : Sensitivity-guided

根据 layer importance 自适应选择位置。

B3 : Adaptive Ticket Search

允许：

- variable ratio ;
- multiple tickets ;
- rollback ;
- regrowth ;
- autonomous stopping。

核心评价：

$$C(M_T)$$

在相同 quality threshold 下谁最小。

17. Stage C : Compression-Frontier Recovery

加入前面的数据方案。

比较：

Recovery Data	含义
None	不恢复
Random	通用随机数据
Parent Hard	Parent 难题
Student Hard	Child 难题
High Gap	Compression-specific real data
Synthetic Frontier	自动生成 frontier data
Oracle	使用测试任务标签，上界

核心希望观察：

$$SyntheticFrontier > HighGap > StudentHard > Random.$$

最重要的科学问题不是 synthetic 一定最好，而是证明：

$$\boxed{UsefulData / } = HardData$$

而是：

$$UsefulData = CompressionSpecificData.$$

18. Stage D : Full Autonomous Self-Compression

Controller 每轮输入：

Current model state
 Current sparsity / memory / latency

 Structural sensitivity
 Capability gap

 Previous action
 Previous recovery gain
 Previous failure

 Accepted history
 Rollback history
 Recovery cost

输出：

Hypothesis

 Candidate tickets

 Expected risk

 Recovery plan

 Stop / continue

执行：

$$Observe \rightarrow Hypothesize \rightarrow Act \rightarrow Evaluate \rightarrow Reflect.$$

最后连续运行：

$$M_0 \rightarrow \dots \rightarrow M_T$$

直至自主停止。

19. Main Ablations

A1. No History

$$\pi(Profile_t)$$

vs.

$$\pi(Profile_t, H_t).$$

验证长期经验是否真正有用。

A2. No Compression Gap

只提供 structural sensitivity。

测试：

$$G_t(x)$$

是否提供额外价值。

A3. No Regrowth

只允许：

$$1 \rightarrow 0.$$

验证 reversible search。

A4. No Rewind

失败 child 继续作为下一代。

测试 cumulative damage。

A5. Random Recovery

High-gap / synthetic :

vs.

random。

A6. External Controller vs Self Controller

External :

$$M_0$$

一直负责控制。

Self :

$$M_t$$

负责产生 M_{t+1} 。

这是整篇论文最关键的实验之一。

A7. Fixed Stop vs Autonomous Stop

例如 :

$$50\% \text{ sparsity}$$

vs.

自动发现 frontier。

20. 一个特别值得主动研究的 Failure Mode

Self-Compression Degeneration

随着 :

$$M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow \cdots ,$$

可能出现：

```
M0:
Task ✓
Controller ✓

M3:
Task ✓
Controller ✓

M5:
Task ✓
Controller △

M7:
Task ✓
Controller ✕
```

即模型还能完成普通 benchmark：

$$TaskPerf(M_t) \approx good,$$

但是已经无法可靠：

- 分析 compression failure；
- 选择下一步 action；
- 判断风险；
- 决定何时停止。

于是存在：

Self-Governance Frontier

并且它不一定和：

Task Performance Frontier

重合。

这可以产生一个很强的研究问题：

Does a model lose the ability to improve itself before it loses the ability to solve tasks?

即使最终 full self-compression 没有完全成功，这个现象本身也可能成为重要分析贡献。

21. Metrics

Compression

$$CR = \frac{C(M_0)}{C(M_T)}.$$

Frontier Quality

固定 performance tolerance :

$$C(M_T^{ours}) < C(M_T^{baseline}).$$

Search Efficiency

记录 :

$$N_{ticket}, N_{rollback}, GPUHours, RecoveryTokens.$$

Recovery Efficiency

$$RE = \frac{Perf(M_{recover}) - Perf(M_{compressed})}{RecoveryCost}.$$

Capability Inheritance

记录 :

$$\mathbf{P}(M_0), \mathbf{P}(M_1), \dots, \mathbf{P}(M_T).$$

Governance

额外设计 :

$$Gov(M_t)$$

衡量 :

- action selection ;
- failure diagnosis ;
- risk prediction ;
- stopping decision.

22. 最重要的 Figures

Figure 1 : Generational Self-Compression

展示：

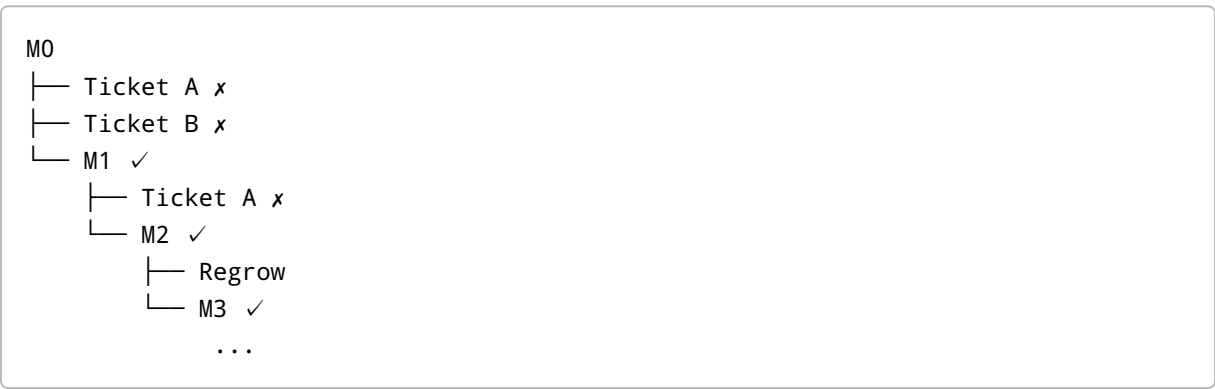
$$Diagnose \rightarrow TicketSearch \rightarrow Compress \rightarrow Recover \rightarrow Inherit.$$

Figure 2 : Compressibility Frontier

比较：

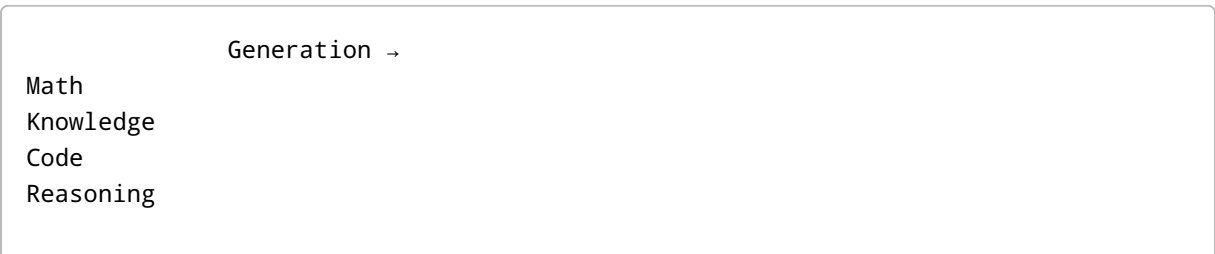
- one-shot ;
- fixed iterative ;
- adaptive ;
- autonomous + recovery。

Figure 3 : Model Lineage



这是最能体现 Lottery Ticket Discovery 的 Figure。

Figure 4 : Capability Frontier Heatmap



Instruction
LongContext

观察每项能力何时出现明显 degeneration。

Figure 5 : Task Frontier vs Governance Frontier

同时画：

$$TaskPerf(M_t)$$

和：

$$Gov(M_t)$$

随 compression 增加的曲线。

23. Code Framework

在前面的 Compression Frontier repo 上升级：

```
autonomous_lottery/  
|  
├─ compression/  
|   ├── wanda.py  
|   ├── sparsegpt.py  
|   ├── structured.py  
|   └─ regrowth.py  
|  
├─ profiling/  
|   ├── layer_sensitivity.py  
|   ├── compression_gap.py  
|   ├── capability_profile.py  
|   └─ governance_profile.py  
|  
├─ ticket/  
|   ├── proposal.py  
|   ├── critic.py  
|   ├── search.py  
|   └─ archive.py  
|  
└─ recovery/
```

```
| | | frontier_mining.py
| | | abstraction.py
| | | generator.py
| | | verifier.py
| | | lora.py
| | | distillation.py
|
| | controller/
| | | policy.py
| | | memory.py
| | | reflection.py
| | | stopping.py
|
| | evaluation/
| | | benchmark.py
| | | capability.py
| | | governance.py
| | | cost.py
|
| | state/
| | | generation.py
| | | lineage.py
| | | experiment_db.py
|
| | scripts/
| | | run_generation.py
| | | run_lineage.py
| | | run_stage_a.sh
| | | run_stage_b.sh
| | | run_stage_c.sh
| | | run_autonomous.sh
```

24. MVP：最小实验

第一版不用跑 24 小时。

只做：

10 ~ 20

个 generation。

Model

Qwen2.5-3B。

Operator

只用一种 pruning。

每轮 Candidate

例如：

Ticket A:
+3% sparsity

Ticket B:
+5% sparsity

Ticket C:
+8% sparsity

同时选择不同 region。

Acceptance

例如：

$$\Delta Aggregate < 2\%$$

且：

$$\max_j \Delta P_j < 5\%.$$

Stop

连续三轮：

all candidate tickets fail

则声明：

$$FrontierFound.$$

25. Go / No-Go Gates

Gate 1

是否存在稳定的 performance cliff / capability frontier ?

如果没有：

暂停 Agent 部分。

Gate 2

Adaptive Search 是否优于 Fixed Iterative ?

需要：

$$CR_{adaptive} > CR_{fixed}.$$

否则：

autonomous search 没有必要。

Gate 3

Compression Gap 是否预测 downstream loss ?

要求：

$$\text{corr}(G_t, \text{DownstreamFailure}) > 0$$

并且：

$$Recovery_{HighGap} > Recovery_{Random}.$$

Gate 4

Synthetic Frontier 是否产生额外价值？

要求：

$$SyntheticFrontier > HighGapReal,$$

或者相同效果使用更少数据。

否则：

synthetic generator 只是 optional component。

Gate 5

Self Controller 是否成立？

比较：

$$Controller = M_0$$

和：

$$Controller = M_t.$$

如果 self controller 很快失效，不需要强行维持“完全自主”。

可以转为：

Persistent meta-controller + autonomous compression lineage,

同时报告 Self-Governance Frontier。

26. 与已有工作的核心差异

项目不能定位成：

LLM agent 自动选择 pruning ratio。

这已经与近期工作高度重叠。

应该强调四点：

1. Search the Limit

已有：

$$Given\ CompressionBudget \rightarrow Maximize\ Performance.$$

我们：

Given PerformanceTolerance $\rightarrow$ Discover Minimum Model.

2. Generational Self-Compression

已有：

Agent_{large} $\rightarrow$ Model.

我们：

$M_t \rightarrow M_{t+1}$.

3. Compression-Aware Recovery

$D_t = f(M_t, M_{t+1}, c_t)$.

数据随着压缩状态改变。

4. Reversible Ticket Search

不仅：

Prune,

而是：

Prune + Rewind + Regrow + Recover.

27. Expected Contributions

Contribution 1 — Compressibility Frontier

提出新的模型压缩问题：

How small can a model make itself under a fixed capability constraint?

Contribution 2 — Autonomous Lottery Ticket Discovery

提出：

Diagnose → *Search* → *Compress* → *Recover* → *Evaluate* → *Inherit*

的长期自主模型压缩框架。

Contribution 3 — Compression-Frontier Recovery

利用：

$$M_t - M_{t+1}$$

的能力差异主动发现和生成最有价值的恢复数据。

Contribution 4 — Self-Governance Frontier

研究：

模型在丧失任务能力之前，是否会首先丧失继续可靠压缩自己的能力？

28. Paper Narrative

不建议写成：

Lottery Ticket + BigBang + Agent + Synthetic Data。

这样故事会过散。

整篇论文围绕一句话：

How small can a model make itself?

然后组织为三层：

Observation

Compression progressively exposes a structured capability frontier.

Method

The model searches for smaller viable descendants through reversible ticket search and compression-aware recovery.

Goal

The process terminates automatically when it reaches the empirical compressibility frontier.

最终形成：

Compress → *Fail* → *Understand* → *Recover* → *Inherit* → *Compress Again*

29. 暂定标题

首选

How Small Can a Model Make Itself? Autonomous Lottery Ticket Discovery for LLM Compression

备选

AutoTicket: Autonomous Discovery of the Compressibility Frontier in Large Language Models

如果 Self-Controller 实验非常强

Self-Compressing Language Models: Generational Search for the Compressibility Frontier

30. 当前最推荐的执行顺序

Step 1
Qwen 3B + single pruning operator
→ 完整 sparsity / capability curve

Step 2
Fixed iterative pruning
vs
adaptive region / ratio
→ 验证 search 是否有价值

Step 3
增加 rollback + regrowth
→ Reversible Ticket Search

Step 4
加入 Compression Gap
→ 找到 capability-specific damage

Step 5
High-gap recovery
vs
random recovery
→ 验证 Compression Frontier

Step 6
Synthetic frontier data
→ BigBang-style Generator/Critic

Step 7
让 Child 接替 Parent
→ Generational Self-Compression

Step 8
持续搜索直到所有 affordable ticket 都失败
→ Empirical Compressibility Frontier

整体原则：

每增加一个智能组件，都必须先证明它解决了前一阶段暴露出的具体问题。

因此，即使最后完整的 autonomous system 没有达到预期，中间仍然可以留下多个独立的科研结论：

1. Compressibility Frontier ；
2. Compression-specific capability degradation ；
3. Reversible ticket search ；
4. Compression-aware recovery ；
5. Self-Governance Frontier。

这使整个项目相比“一次性实现一个复杂 Agent”具有更低的研究风险。